

Internship Project Report

EDGE-BASED SMART SUSPENSION SYSTEM

Jampani Rohith

[GitHub/jampanirohith/Edge-Based-Smart-Suspension](https://github.com/jampanirohith/Edge-Based-Smart-Suspension)

ABSTRACT

This project develops a real-time edge-based system for detecting road surface anomalies — potholes and speed bumps — using a single monocular camera and a YOLOv8 object detection model. Upon detection, a ground-plane projection algorithm estimates the distance to each hazard, and a geometric risk metric quantifies its severity. Time-to-Impact (TTI) is calculated using the estimated distance and vehicle speed. A rule-based decision engine combines severity and urgency to produce suspension directives — NORMAL, PREPARE, or ADJUST NOW — before the vehicle reaches the obstacle. All processing runs locally on an edge device, removing the need for cloud infrastructure. The system offers a cost-effective alternative to the predictive suspension technologies found in premium vehicles.

TABLE OF CONTENTS

Section	Page
ABSTRACT	2
CHAPTER 1: INTRODUCTION	4
CHAPTER 2: LITERATURE REVIEW	5
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	6
CHAPTER 4: METHODOLOGY	8
CHAPTER 5: IMPLEMENTATION	10
CHAPTER 6: RESULTS AND DISCUSSION	11
CHAPTER 7: CONCLUSION AND FUTURE SCOPE	14
REFERENCES	15
APPENDICES	16

CHAPTER 1: INTRODUCTION

1.1 Background

Road irregularities such as potholes and speed bumps cause sudden suspension stress, accelerate mechanical wear, and reduce passenger comfort. Standard suspension systems only react after contact with a hazard — there is no warning or preparation beforehand.

Premium vehicles from BMW and Mercedes-Benz offer predictive suspension systems using cameras and radar, but these rely on expensive hardware that is out of reach for most vehicles. Recent advances in deep learning and edge computing make it possible to build a similar capability at a fraction of the cost using a single camera and a small compute device.

1.2 Problem Statement

Drivers encounter road anomalies with little to no warning, causing abrupt suspension impacts that contribute to component wear and passenger discomfort. Existing low-cost road monitoring solutions detect hazards but do not estimate distance, assess severity, or generate control directives — leaving a gap between detection and actionable suspension response.

1.3 Aim of the Project

To develop an edge-based computer vision framework that detects potholes and speed bumps in real time and generates predictive suspension adjustment commands before the vehicle reaches the obstacle.

1.4 Objectives

- Compile and annotate a dataset of pothole and speed bump images.
- Train a YOLOv8 object detection model to identify road anomalies.
- Implement real-time inference on local edge hardware.
- Estimate hazard distance using monocular ground-plane projection.
- Classify anomaly severity based on estimated physical width.
- Compute Time-to-Impact (TTI) using estimated distance and vehicle speed.
- Generate suspension directives (NORMAL / PREPARE / ADJUST NOW) using a rule-based decision engine.

1.5 Proposed Methodology

A forward-facing monocular camera streams road footage to a local edge device. A YOLOv8n model processes each frame to detect potholes and speed bumps. Ground-plane geometry converts pixel coordinates to metric distances. Physical width is derived from the bounding box and estimated distance to grade severity. TTI is calculated using the vehicle speed. A decision engine fuses these inputs to output a suspension directive per frame.

1.6 Scope of the Project

The project covers real-time road hazard detection, distance estimation, severity classification, and decision generation within an edge-computing environment. Physical suspension actuation, CAN bus integration, and embedded hardware deployment are out of scope for this internship phase and are reserved for future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems

Road condition monitoring has traditionally relied on manual inspection or vehicle-mounted sensors (accelerometers, gyroscopes) that detect anomalies reactively — only after the vehicle has hit the hazard. Premium automotive manufacturers have introduced camera and radar-based predictive suspension systems, but the sensor hardware and proprietary ECUs involved push costs well beyond the affordable range.

Deep learning-based object detection — particularly YOLO-family models — has made real-time, camera-based pothole and speed bump detection feasible on standard hardware. Edge computing platforms now provide the low latency needed for in-vehicle inference without cloud dependency.

2.2 Research Papers Survey

The primary reference for this project is SBP-YOLO (Liang et al., 2026) [1], which proposes a lightweight YOLOv11-based detector specifically for speed bumps and potholes to drive predictive suspension control. The model achieves 87% mAP at 139 FPS on an NVIDIA Jetson AGX Xavier. POT-YOLO (Bhavana et al., 2024) [2] achieves 97.6% precision on pothole classification using a YOLOv8-based edge architecture, validating the approach taken in this project.

2.3 Research Gap

Most published solutions focus solely on anomaly detection without integrating spatial estimation, severity grading, or suspension-oriented decision support. Many also depend on expensive sensors (LiDAR, stereo cameras) or cloud inference, which limits their practical applicability. This project addresses the gap by combining detection, distance estimation, severity assessment, TTI computation, and decision generation in a single monocular edge-based pipeline — without cloud dependency or expensive sensors.

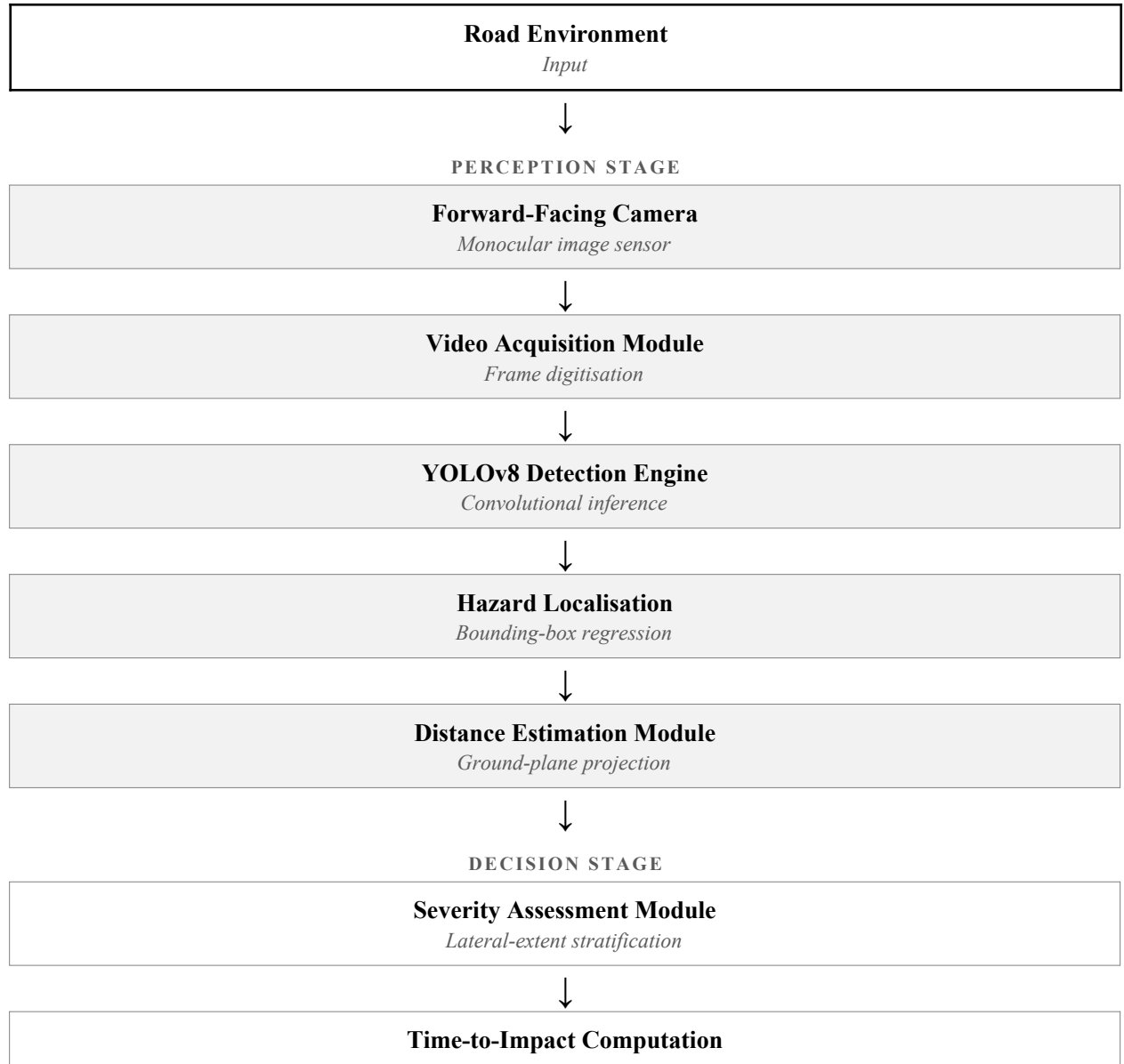
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Proposed System

The proposed system processes live road footage from a forward-facing camera through a sequential pipeline: YOLOv8n detection, ground-plane distance estimation, severity classification, TTI computation, and a rule-based decision engine. All computation runs locally on an edge device. The outputs are an annotated video, a structured CSV log, and a per-frame suspension directive.

3.2 System Architecture

The system is organised into two functional stages. The perception stage handles frame acquisition, object detection, hazard localisation, and distance estimation. The decision stage covers severity assessment, TTI calculation, decision fusion, and output generation. Figure 3.1 illustrates the full module-level architecture.



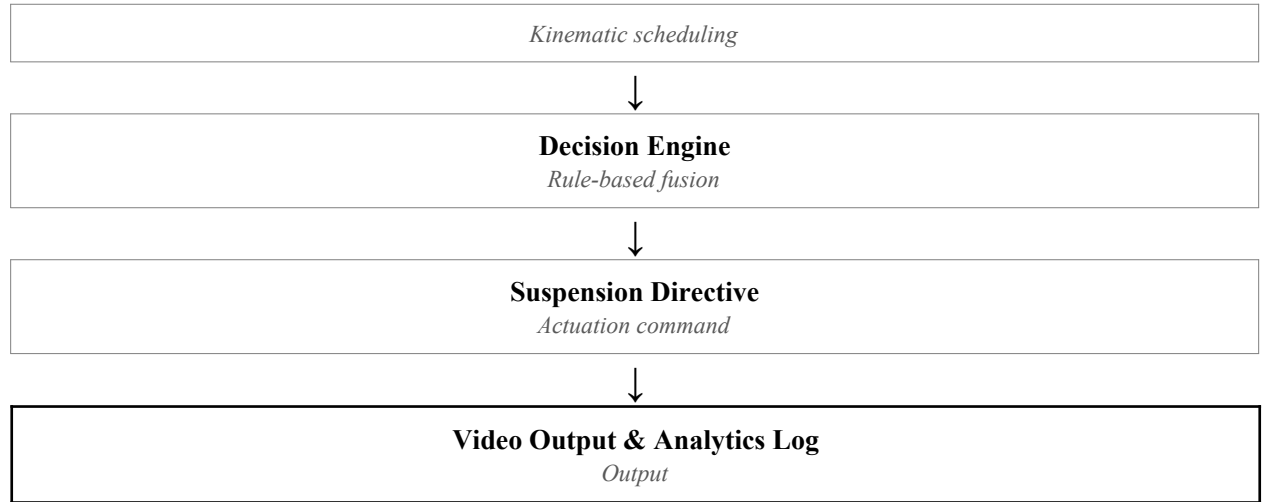


Fig. 3.1. Module-level architecture of the perception-to-actuation pipeline.

3.3 Data Flow

Table 3.1. Stage-wise data-flow pipeline.

Stage	Designation	Description
1	Data Acquisition	Camera captures road scene; OpenCV extracts frames sequentially.
2	Hazard Detection	YOLOv8n returns bounding boxes, class labels, and confidence scores.
3	Spatial Estimation	Bottom edge of bounding box is projected onto the ground plane to estimate range.
4	Severity Analysis	Physical width is derived from box dimensions and range; severity is classified.
5	Impact Assessment	TTI is computed from estimated distance and vehicle speed.
6	Decision Generation	Severity and urgency are fused to produce the suspension directive.
7	Output	Annotated frames are written to video; per-frame records are appended to CSV.

3.4 System Requirements

3.4.1 Hardware Requirements

Table 3.2. Hardware configuration used in the experiment.

Component	Specification	Role
Vehicle-Mounted Camera	Smartphone camera (dashboard mount)	Records forward road footage for system input
Video Transfer	USB cable / microSD card transfer	Transfers footage from camera to compute device
Compute Device	Laptop — NVIDIA GeForce RTX 5050 GPU (CUDA 12.8)	Runs all inference and decision logic
Local Storage	SSD	Stores dataset, model weights, output video, CSV logs

Note: A live wireless video link was not used in this phase. The system was tested on pre-recorded footage. Network streaming is planned for the next phase.

3.4.2 Software Requirements

Table 3.3. Software environment.

Tool / Library	Purpose
Python 3.11	Primary programming language
PyTorch (CUDA 12.8 nightly)	Deep learning backend — required for RTX 5050
Ultralytics YOLOv8	Object detection model and training framework
OpenCV 4.10.0	Video capture, frame processing, annotation, output encoding
Roboflow	Dataset management, annotation, augmentation, YOLO-format export
NumPy / Pandas / CSV	Post-processing and analytical logging
VS Code / Jupyter	Development environment

3.5 Environment Set-Up

The following procedure was used to configure the development environment on Windows 11. The RTX 5050 Laptop GPU requires a CUDA 12.8 nightly build of PyTorch — standard stable builds do not support this GPU.

```
# Create and activate virtual environment
py -3.11 -m venv suspension_env
.\suspension_env\Scripts\Activate

# Install GPU-enabled PyTorch (RTX 5050 requires CUDA 12.8 nightly)
pip install --pre torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/nightly/cu128

# Install remaining dependencies
pip install ultralytics opencv-python==4.10.0.84 roboflow pandas numpy

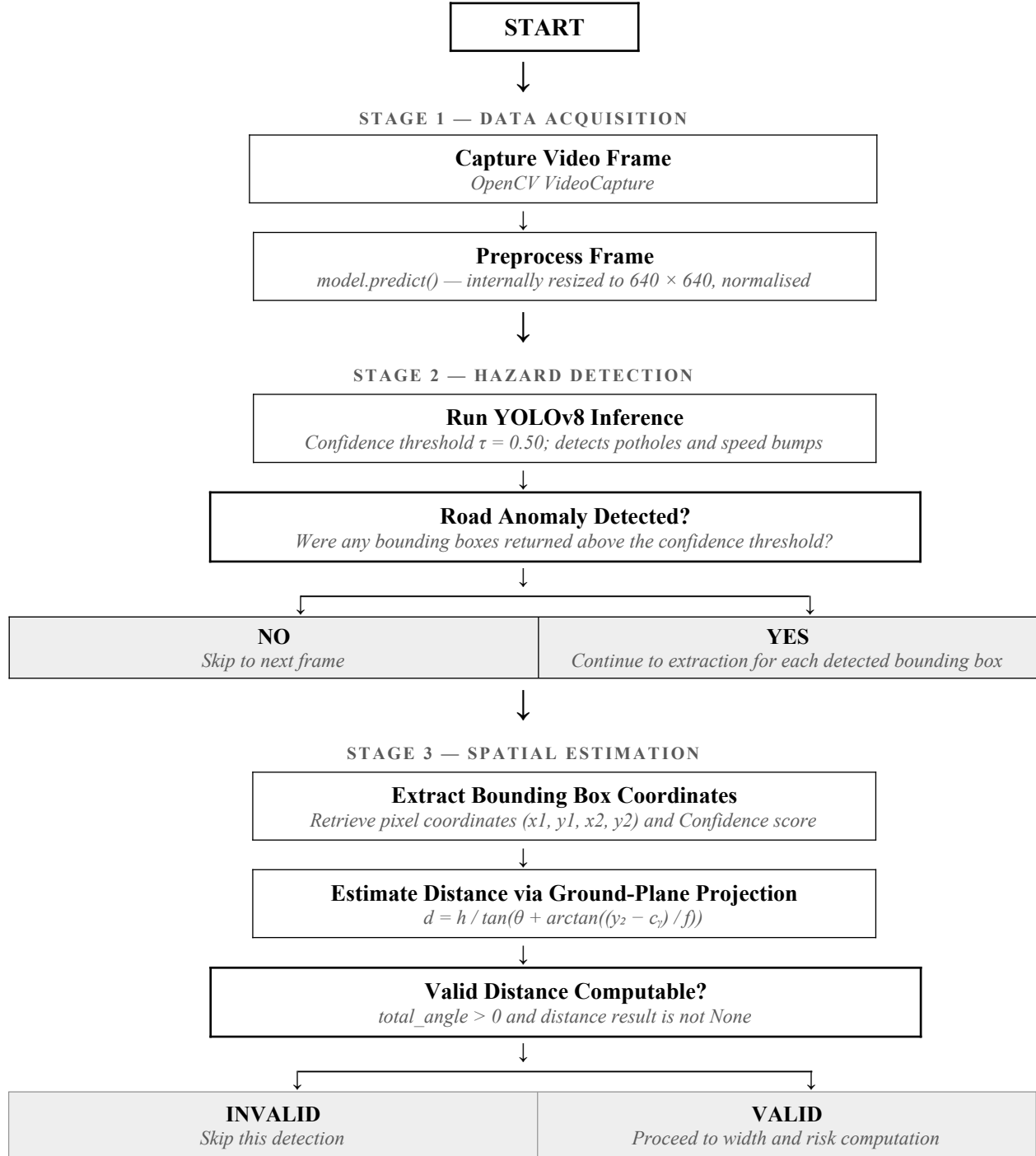
# Verify GPU
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
# Expected: True NVIDIA GeForce RTX 5050 Laptop GPU
```

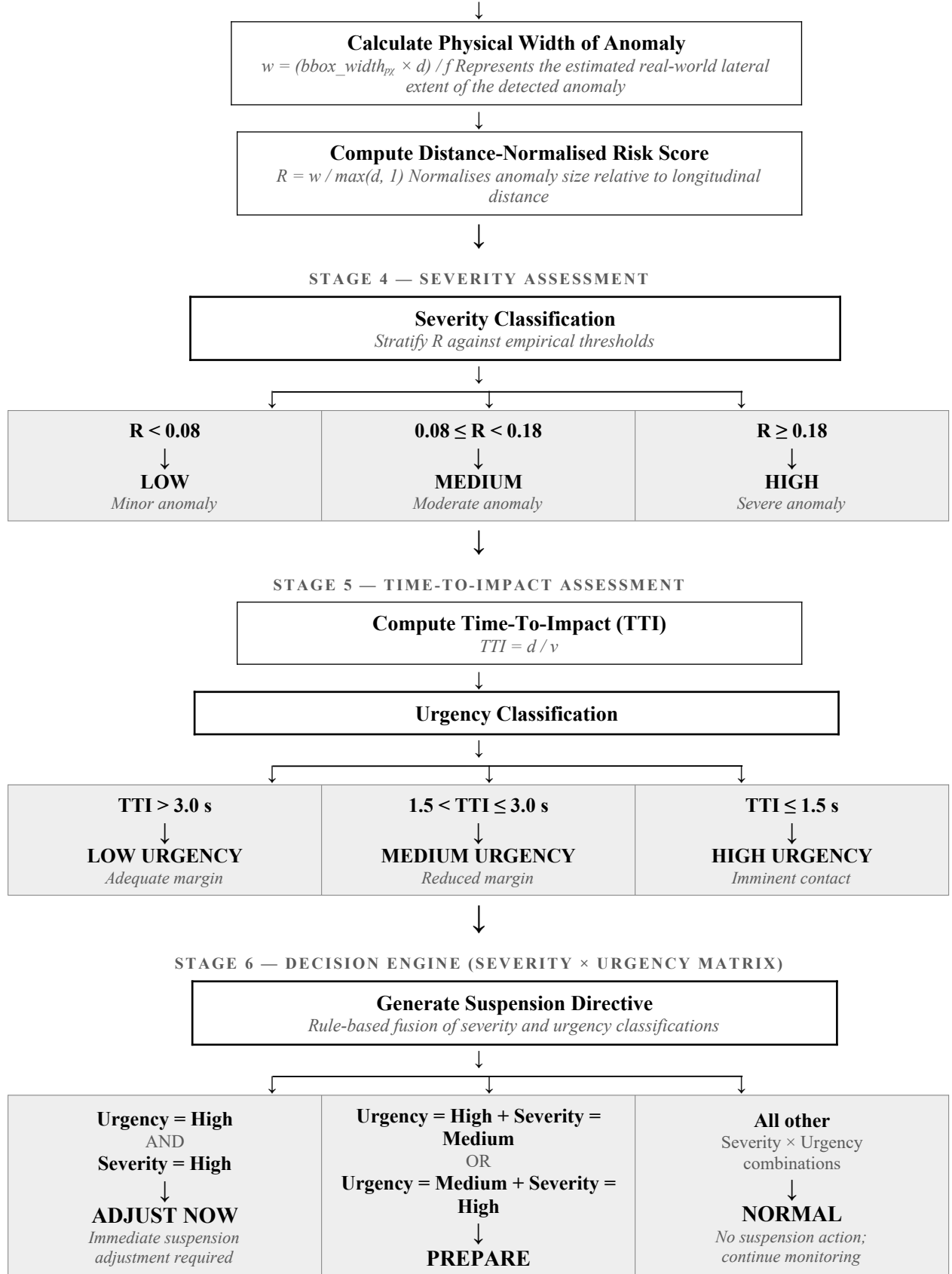
Listing 3.1. Environment configuration and dependency installation.

CHAPTER 4: METHODOLOGY

4.1 System Workflow

Each video frame passes through the complete pipeline sequentially: detection, spatial estimation, severity and TTI computation, decision fusion, and output. Figure 4.1 shows the detailed flowchart including all conditional branches and decision outcomes (NORMAL, PREPARE, ADJUST NOW).





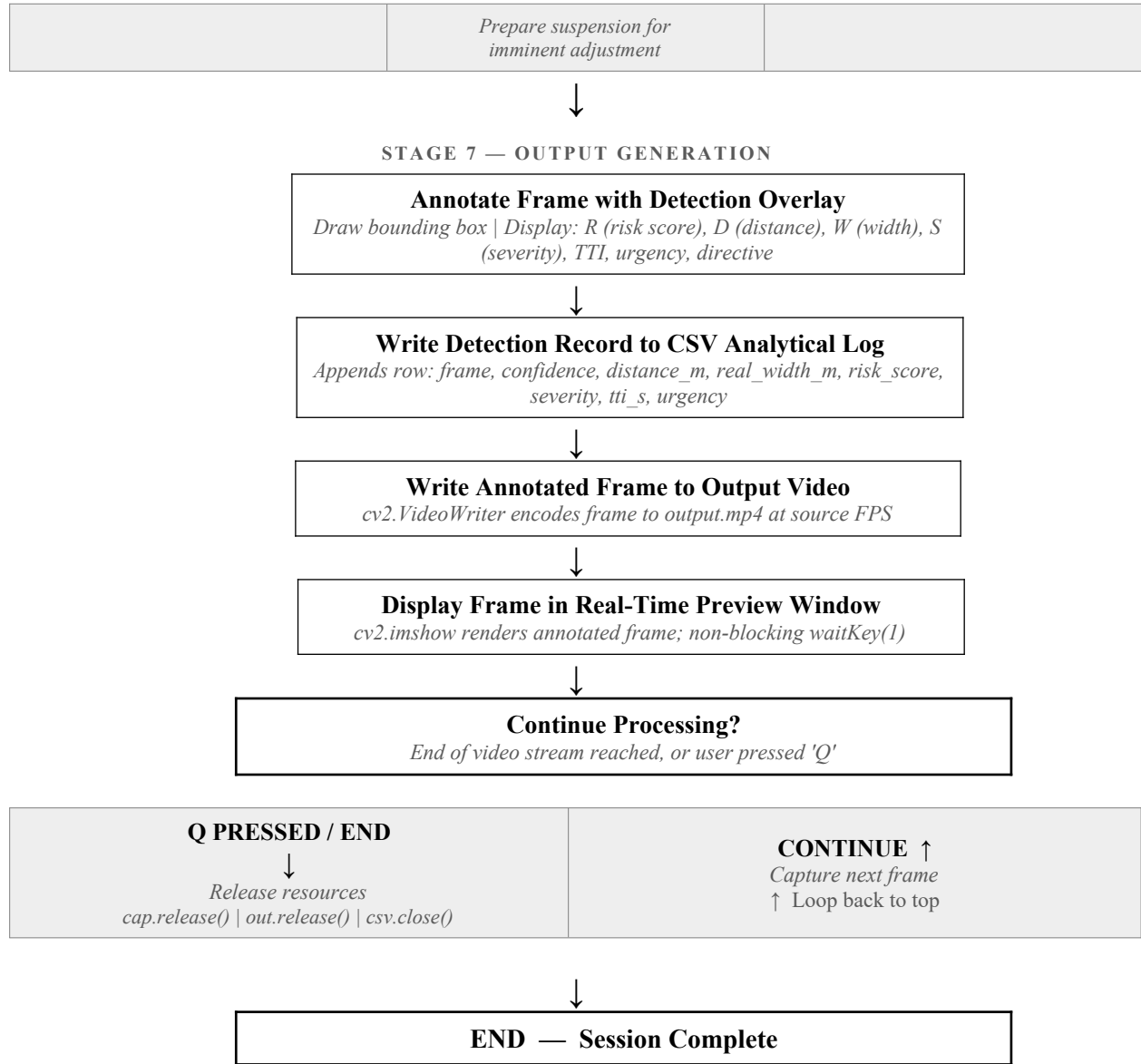


Fig. 4.1. Complete system workflow with all decision branches.

4.2 Dataset Preparation

A unified dataset was assembled from publicly available pothole and speed bump repositories on Roboflow. Images were annotated with bounding boxes in YOLO format. Two classes were defined: Class 0 — Pothole, Class 1 — Speed Bump. The dataset was split 70% training / 11% validation / 19% testing.

4.3 Object Detection Module

The YOLOv8n (nano) model was selected for its low parameter count (3.2M) and fast inference. Transfer learning from COCO pre-trained weights was used to improve convergence speed and generalisation across varied road conditions. For each detected anomaly, the model returns bounding box coordinates (x_1 , y_1 , x_2 , y_2), the class label, and a confidence score.

4.4 Distance Estimation Module

Longitudinal distance is estimated by back-projecting the lower bounding-box edge onto a planar ground model using the camera's extrinsic parameters — mounting height H and pitch angle α :

$$D = H / \tan(\theta), \text{ where } \theta = |\alpha| + \arctan((y_2 - c_y) / f_x)$$

$H = 1.20$ m (camera height), $\alpha = 8.0^\circ$ (pitch), c_y = image vertical centre, f_x = focal length in pixels.

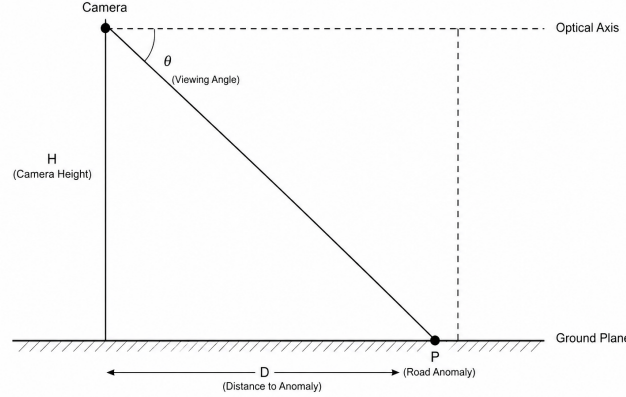


Fig. 4.4.1. Ground-plane perspective projection model.

4.5 Severity Assessment Module

Physical width of the anomaly is estimated from the pixel-domain bounding-box width and the estimated distance:

$$W = (W_{px} \times D) / f_x$$

W is then normalised by distance to give a risk score $R = W / \max(D, 1)$, and mapped to three severity levels:

Table 4.5.1. Structural anomaly severity classification.

Real-World Width (W)	Severity	Implication
< 0.5 m	Low	Minor defect; minimal chassis impact
0.5 m – 1.0 m	Medium	Moderate anomaly; damping adjustment recommended
> 1.0 m	High	Large defect; immediate suspension response required

4.6 Time-To-Impact Analysis

TTI is the time remaining before the front wheel contacts the detected hazard:

$$TTI = D / V, \text{ where } V = v_{km/h} / 3.6$$

TTI is then mapped to an urgency level to prioritise the control response:

Table 4.6.1. TTI urgency classification.

TTI Threshold	Urgency Level	System Implication
> 2 s	Low	Sufficient time; monitor only
1 – 2 s	Medium	Hazard approaching; prepare suspension
< 1 s	High	Imminent; actuate immediately

4.7 Decision Engine

The decision engine cross-references severity and urgency to generate the suspension directive:

Table 4.7.1. Suspension decision matrix.

Urgency	Severity	Directive
High	High	ADJUST NOW — immediate damping modulation
High	Medium	PREPARE — prime suspension for adjustment
Medium	High	PREPARE — begin adjustment sequence
Any other combination	—	NORMAL — continue monitoring

4.8 Model Training

The YOLOv8n model was fine-tuned over 30 epochs on the compiled dataset using the configuration in Table 4.8.1.

Table 4.8.1. Training configuration.

Parameter	Value
Base architecture	YOLOv8n (3.2M parameters)
Initialisation	COCO pre-trained weights
Epochs	30
Batch size	16
Input resolution	640×640
Optimiser	AdamW ($lr_0 = 0.001$)
Confidence threshold	0.50
Augmentation	Horizontal flip, HSV brightness, Mosaic

CHAPTER 5: IMPLEMENTATION

The system is implemented as a single Python script (main.py) that runs all modules in sequence for each video frame. Key code snippets for each module are shown below; full listings are in Appendix C.

5.1 Video Acquisition

```
cap = cv2.VideoCapture(video_path)
ret, frame = cap.read()
```

Code Snippet 5.1.1. Frame extraction using OpenCV VideoCapture.

5.2 Object Detection

```
model = YOLO('best.pt')
results = model.predict(frame, conf=0.5, verbose=False)
```

Code Snippet 5.2.1. YOLOv8n model load and inference.

5.3 Distance Estimation

```
distance = estimate_distance_groundplane(
    y_bottom=y2, image_height=frame.shape[0], focal_px=FOCAL_PX)
```

Code Snippet 5.3.1. Ground-plane distance calculation call.

5.4 Severity Assessment

```
real_width = (bbox_width * distance) / FOCAL_PX
risk_score = real_width / max(distance, 1)
if risk_score < 0.08: severity = 'Low'
elif risk_score < 0.18: severity = 'Medium'
else: severity = 'High'
```

Code Snippet 5.4.1. Width estimation and severity classification.

5.5 Time-To-Impact Analysis

```
tti = distance / (SPEED_KMH / 3.6)
urgency = 'High' if tti < 1.5 else 'Medium' if tti < 3.0 else 'Low'
```

Code Snippet 5.5.1. TTI computation.

5.6 Decision Engine

```
if urgency == 'High' and severity == 'High': decision = 'ADJUST NOW'
elif urgency in ('High', 'Medium') and severity in ('High', 'Medium'):
    decision = 'PREPARE'
else: decision = 'NORMAL'
```

Code Snippet 5.6.1. Rule-based decision engine.

5.7 Output Generation

```
out.write(annotated_frame) # annotated video
writer.writerow([frame_id, cls, distance, width, severity, tti, decision]) # CSV
```

Code Snippet 5.7.1. Output video writing and CSV logging.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Experimental Setup

The system was tested on a pre-recorded road video within a software-in-the-loop (SIL) setup. Table 6.1 documents the hardware, software, and camera calibration parameters.

Table 6.1. *Experimental environment and sensor calibration.*

System Attribute	Value
Hardware	
GPU	NVIDIA GeForce RTX 5050 Laptop GPU (CUDA 12.8)
RAM	16 GB DDR5
Software	
Framework	Python 3.11 / PyTorch (nightly) / Ultralytics YOLOv8
Vision Library	OpenCV 4.10.0
Camera Calibration	
Mounting Height (H)	1.20 m
Focal Length (f_x)	650.0 px
Pitch Angle (α)	8.0° downward
Input Resolution	1280 × 720 (processed at 640 × 640)

6.2 Dataset Statistics

Table 6.2. *Dataset split.*

Partition	Images
Training	3046
Validation	468
Total	3514

6.3 Training Convergence

The model was fine-tuned for 30 epochs. Loss curves showed steady convergence without overfitting. Final validation metrics are in Table 6.3.

Table 6.3. *Validation metrics.*

Metric	Value
Precision	0.798
Recall	0.644
mAP@50	0.731
mAP@50–95	0.415

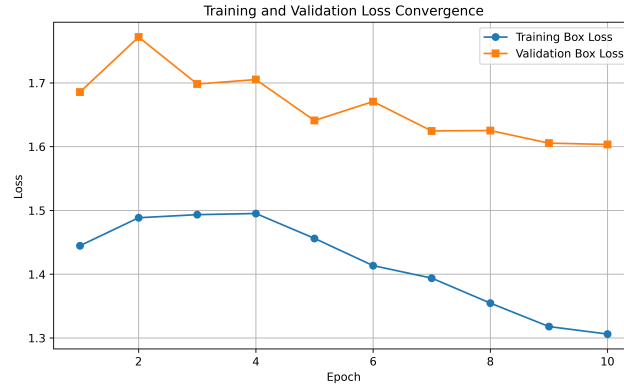


Fig. 6.1. Training and validation loss convergence over 30 epochs.

6.4 Class-Wise Performance

Table 6.4. Per-class detection performance.

Class	Precision	Recall	mAP@50	mAP@50–95
Speed Bump	0.904	0.745	0.852	0.503
Pothole	0.692	0.543	0.610	0.326

Speed bumps achieved higher accuracy due to their consistent geometry and painted markings. Potholes vary significantly in shape, size, and surface texture, making feature extraction harder.

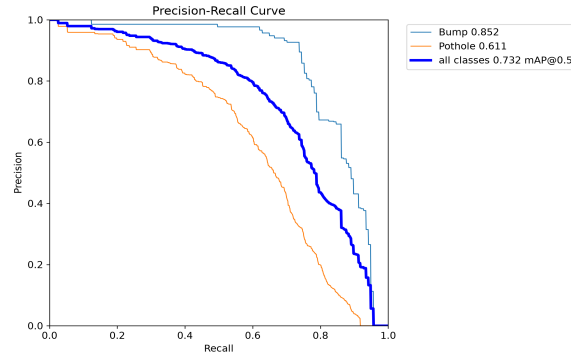


Fig. 6.2. PR convergence curve for the trained model.

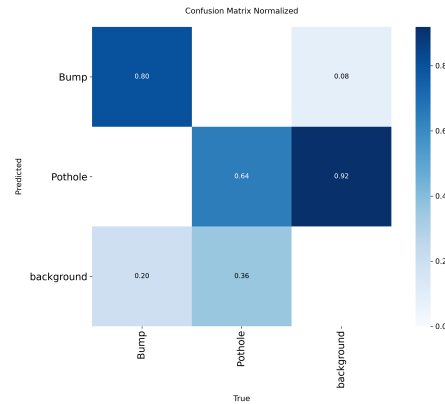


Fig. 6.3. Validation confusion matrix.

6.5 Detection Results

The model detected both anomaly classes reliably across the evaluation sequence:

- Real-time localisation of potholes and speed bumps across the full camera field of view.
- Simultaneous multi-object detection within a single frame.
- No frame drops at the target inference rate.



Fig. 6.4. Output frame showing pothole bounding box and confidence score.

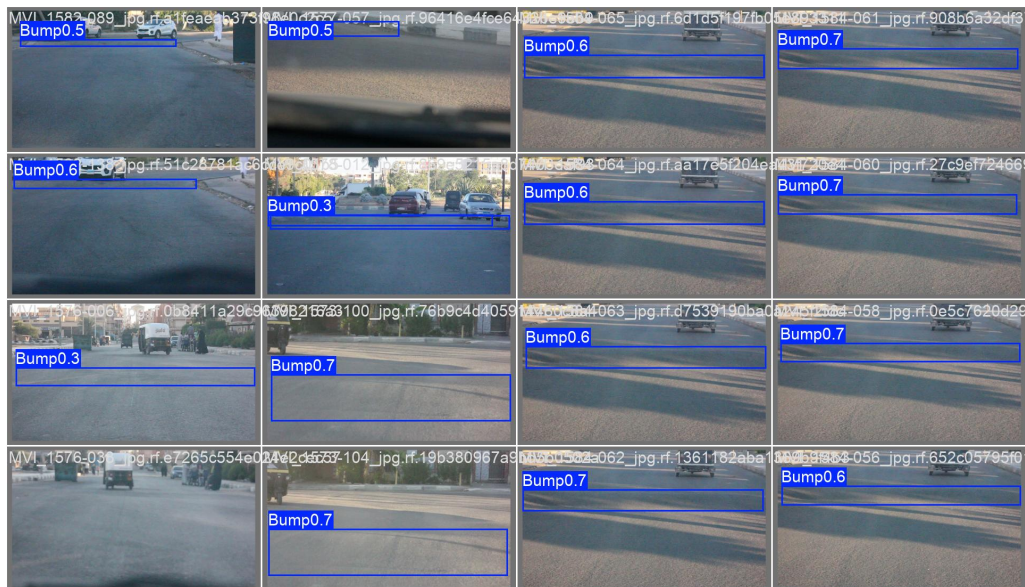


Fig. 6.5. Output frame showing speed bump detection.

6.6 Distance Estimation Results

Table 6.5. Sample distance estimation outputs.

Detection	Estimated Distance
Pothole A	2.8 m
Pothole B	4.5 m
Speed Bump A	6.2 m
Speed Bump B	8.7 m

6.7 Severity Assessment Results

Table 6.6. Sample severity outputs.

Distance	Width (W)	Severity
3.2 m	0.34 m	Low
4.5 m	1.25 m	Medium
6.8 m	3.45 m	High

6.8 Time-to-Impact Analysis

Table 6.7. TTI at 40 km/h.

Distance	Computed TTI
5 m	0.45 s
10 m	0.90 s
15 m	1.35 s
20 m	1.80 s

6.9 Suspension Decision Results

Table 6.8. Sample decision engine outputs.

Severity	TTI	Directive
Low	2.5 s	NORMAL
Medium	1.4 s	PREPARE
High	0.7 s	ADJUST NOW



Fig. 6.7. Severity assessment and directive overlay on a sample frame.

6.10 Runtime Logs

Table 6.9. Sample CSV log output.

Frame	Class	Dist. (m)	Width (m)	Severity	Speed (kph)	TTI (s)	Directive
1042	None	N/A	N/A	N/A	45.0	N/A	NORMAL
1043	Pothole	18.5	0.85	Medium	45.0	1.48	PREPARE
1044	Pothole	12.2	0.88	Medium	45.0	0.97	ADJUST NOW

```
[DEBUG] Frame 1043: Pothole | Conf=0.82 | Dist=18.5m | Width=0.85m | Sev=Medium
[CALC] TTI = 18.5 / 12.5 = 1.48s -> Urgency: Medium
[ACTION] Directive: PREPARE

[DEBUG] Frame 1044: Pothole | Conf=0.86 | Dist=12.2m | Width=0.88m | Sev=Medium
[CALC] TTI = 12.2 / 12.5 = 0.97s -> Urgency: High
[ACTION] Directive: ADJUST NOW
```

Console execution trace (abbreviated).

6.11 Computational Throughput

Table 6.10. Pipeline processing performance.

Metric	Value
Inference latency	1.2 ms / frame
Post-processing (TTI / severity)	0.9 ms / frame
Total pipeline latency	~2.1 ms / frame
Sustained throughput	> 45 FPS
External network dependency	None

6.12 Discussion

The pipeline ran at over 45 FPS with a total per-frame latency of ~2.1 ms, well within the real-time requirements for automotive applications. The YOLOv8n model detected both road anomaly classes reliably across the test sequence, and the downstream distance, severity, and TTI modules produced consistent outputs frame by frame.

The main limitation is that the ground-plane distance model assumes a flat road and fixed camera orientation. In practice, road gradients or camera vibration will introduce error into the estimated distances, which then propagates into the severity and TTI calculations.

The fixed vehicle speed of 40 km/h is another constraint — in a real deployment, live speed data from GPS or the vehicle CAN bus would be needed. Despite these limitations, the system demonstrates a working proof-of-concept for vision-guided predictive suspension control on standard laptop hardware.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This internship project built and validated a complete edge-based pipeline for predictive vehicle suspension control. Using a single monocular camera and YOLOv8n, the system detects potholes and speed bumps, estimates their distance and physical severity, computes time-to-impact, and generates suspension directives in real time. Key outcomes:

- YOLOv8n fine-tuned on 3,514 images — mAP@50: 0.731, Precision: 0.798, Recall: 0.644.
- Ground-plane projection estimates distance using only camera height and pitch angle.
- Risk-score metric classifies hazard severity as Low, Medium, or High.
- TTI computed per frame using estimated distance and fixed vehicle speed (40 km/h).
- Rule-based decision engine outputs NORMAL / PREPARE / ADJUST NOW.
- Pipeline sustains > 45 FPS with ~2.1 ms total latency per frame on RTX 5050.
- Fully local execution — no cloud dependency or network required.

The system provides a cost-effective, software-validated starting point for integrating predictive suspension control into standard vehicles.

7.2 Limitations

- Distance estimation assumes a flat road surface and fixed camera orientation — deviations cause error.
- Monocular vision cannot directly measure pothole depth.
- Vehicle speed is hardcoded at 40 km/h; no live speed input is integrated.
- Decision logic uses fixed thresholds and does not adapt to varying road materials or vehicle load.
- Validation was performed on pre-recorded video only — no physical actuation was tested.
- Detection accuracy drops under rain, low light, or camera contamination.

7.3 Future Scope

- Depth Estimation: Integrate Depth Anything or MiDaS for per-pixel depth from a single camera.
- Embedded Deployment: Deploy on NVIDIA Jetson Orin Nano or Raspberry Pi with Hailo-8L accelerator.
- Suspension Hardware: Translate directives into PWM signals for electronically controlled dampers.
- CAN Bus Integration: Read live speed, steering angle, and load from vehicle ECUs.
- Multi-Sensor Fusion: Combine camera with IMU (for pitch compensation) and LiDAR.
- Adaptive Control: Replace fixed-threshold decision logic with a reinforcement-learning policy.
- Simulation: Validate actuation strategies in MATLAB/Simulink or Unreal Engine before real-world testing.
- Live Streaming: Implement 5G/Wi-Fi video link to a shared edge server for multi-vehicle deployment.

REFERENCES

- [1] C. Liang, J. Yuan, L. Gou, L. Luo, J. Fu, and M. Yu, “SBP-YOLO: A lightweight real-time model for detecting speed bumps and potholes toward intelligent vehicle suspension systems,” *Journal of Real-Time Image Processing*, vol. 23, no. 52, 2026.
- [2] G. Jocher et al., “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] Ultralytics, “YOLOv8 Documentation,” 2023. [Online]. Available: <https://docs.ultralytics.com>
- [4] Roboflow, “Potholes-Bumps Dataset,” 2024. [Online]. Available: <https://universe.roboflow.com/jampanis-workspace/potholesbumps>
- [5] OpenCV, “Camera calibration and 3D reconstruction,” 2024. [Online]. Available: <https://docs.opencv.org/4.x/d9/d0c>

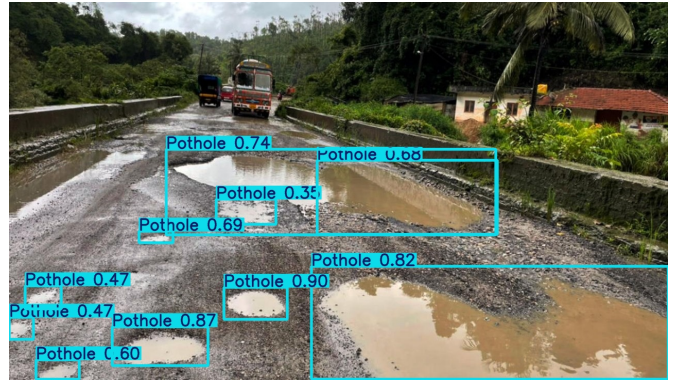
APPENDICES

Appendix A — Dataset Samples

Representative annotated training images from the Roboflow corpus.



Annotated sample — Speed Bump class.



Annotated sample — Pothole class.

Appendix B — Model Training Configuration

Table B.1. YOLOv8n training hyperparameters.

Parameter	Value	Notes
Base architecture	YOLOv8n	Nano variant; 3.2M parameters
Initialisation	COCO pre-trained	Transfer learning (yolov8n.pt)
Epochs	30	Early stopping on validation plateau
Batch size	16	Adjusted for VRAM
Input resolution	640×640	Standard YOLO resolution
Optimiser	AdamW	$lr_0 = 0.001$, weight decay = 5×10^{-4}
Confidence threshold	0.50	Inference-time filter
NMS IOU threshold	0.45	Duplicate suppression limit
Augmentation	Flip 0.5, HSV 0.4, Mosaic	Applied during training
Workers	0 (Windows)	Prevents multiprocessing deadlock

Appendix C — Source Code Listings

Listing C.1 — Ground-Plane Distance Estimation

```
def estimate_distance_groundplane(y_bottom, image_height, focal_px):
    pitch_rad = math.radians(PITCH_DEG) # PITCH_DEG = 8.0
    cy = image_height / 2
    angle_rad = math.atan((y_bottom - cy) / focal_px)
    total_angle = pitch_rad + angle_rad
    if total_angle <= 0: return None
    return round(CAMERA_HEIGHT_M / math.tan(total_angle), 2)
```

Listing C.1. Ground-plane projection function (main.py).

Listing C.2 — Severity Classification

```
def classify_severity(bbox_width_px, distance_m, focal_px):
    real_width = (bbox_width_px * distance_m) / focal_px
    risk_score = real_width / max(distance_m, 1)
    if risk_score < 0.08: return real_width, risk_score, 'Low'
    elif risk_score < 0.18: return real_width, risk_score, 'Medium'
    else: return real_width, risk_score, 'High'
```

Listing C.2. Width estimation and severity classification (main.py).

Listing C.3 — TTI Computation

```
def compute_tti(distance_m, speed_kmh=40):
    speed_mps = speed_kmh / 3.6
    tti_s = distance_m / speed_mps
    if tti_s > 3.0: urgency = 'Low'
    elif tti_s > 1.5: urgency = 'Medium'
    else: urgency = 'High'
    return round(tti_s, 2), urgency
```

Listing C.3. Time-to-impact computation (main.py).

Listing C.4 — Decision Engine

```
def generate_directive(severity, urgency):
    if urgency == 'High' and severity == 'High': return 'ADJUST NOW'
    elif urgency == 'High' and severity == 'Medium': return 'PREPARE'
    elif urgency == 'Medium' and severity == 'High': return 'PREPARE'
    else: return 'NORMAL'
```

Listing C.4. Rule-based decision engine (main.py).

Appendix D — Experimental Outputs

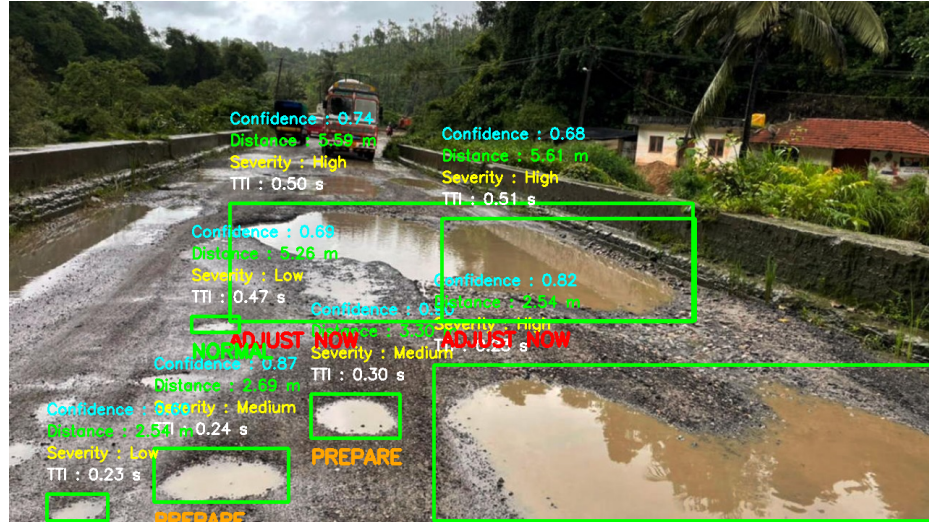


Fig. D.1. Pothole detection with bounding box, distance, severity and directive overlay.

Table D.1. Extended CSV log sample.

Frame	Class	Dist. (m)	Width (m)	Severity	Speed (kph)	TTI (s)	Directive
1042	None	N/A	N/A	N/A	45.0	N/A	NORMAL
1043	Pothole	18.5	0.85	Medium	45.0	1.48	PREPARE
1044	Pothole	12.2	0.88	Medium	45.0	0.97	ADJUST NOW
1045	Speed Bump	9.3	1.42	High	45.0	0.74	ADJUST NOW
1046	None	N/A	N/A	N/A	45.0	N/A	NORMAL